*Document number: P3318R0*
*Audience: LEWG*

*Ville Voutilainen*
*2024-05-22*

---

# Throwing violation handlers, from an application programming perspective

## Abstract

This paper is an amicus brief for the LEWG noexcept policy discussion. It makes the case that for application programming, standard library functions with narrow contracts should not be noexcept, and we should keep the earlier policies about that, i.e. keep or if necessary, reinstate, The Lakos Rule.

It has nothing whatsoever to do with any testing.

What it is about is allowing programmers to handle contract violations programmatically, and recover from them. Because the business rules of their domain deem that the right thing to do, and terminating is unattractive, and wrong for those business rules.

In a nutshell,

```
void whatever_func() {
    try {
        some_func_with_preconditions(oink, boink);
    } catch (const MyOutOfRangeContractException& e) {
        log(e);
        disable_offending_subsystem();
        resume_normal_operation();
    }
}
```

## "But you can't do that, a contract violation means that the program is in a catastrophic and unrecoverable state"

No, that's not at all what it means. A checked contract means the complete opposite of that; contracts define what the UB of a contract violation means. The check intercepts the offending operation before the actual UB occurs. You may have inadvertent null pointers passed to functions that require non-null pointers, you may have inverted logical conditions, arguments passed in the wrong order, and all such things that are typos or thinkos, things that non-expert programmers often write in a buggy manner. And we can recover from all that, especially if those bugs are in parts of the program that are used 2% of the time with 0.5% importance, and we can keep the rest of the program running, instead of abruptly terminating all of it.

## "But you can't do that, a contract violation means that you don't *know* that the program isn't in a catastrophic and unrecoverable state"

That may be a business rule for some domains. It's not the rule in some others. I don't need to know that, my architectural choice is to limp along hoping that the state isn't truly catastrophic. That is the correct choice for some of my domains, and superimposing the requirements of other domains as language rules is rather untoward.

## "But you shouldn't do that, you're exposing a lot of code to an attacker"

No, I'm not. I'm not going to do this in security-sensitive parts of my whole application, and I'm not going to put such parts into the same process. But I do need to make the best effort that the process running my main UI doesn't crash, and it's not feasible to split the main UI into yet another separate process. Some examples like "an online sudoku game" do not run into the suggested attacker problem; the parts that interface with the internets are separated into separate service processes, and there's platform security facilities stronger than any of you can break between the main UI process and the separate services.

## So, okay, fine, what does this actually have to do with LEWG and its noexcept policy?

Please go back to the pseudo-example in the abstract, and consider what happens if some_func_with_preconditions() is a C++ standard library function.

If we introduce a policy where functions with narrow contracts but with "Throws: nothing" semantics are made noexcept, the technique illustrated will not work. A throwing contract violation handler will not work, the noexcept boundary will terminate the application. A precondition turned into a C++26(?) Contract by the library implementation cannot be turned into all the things a contract can be turned into, it can't be turned into an exception that is handled by the program.

I'm suggesting that we shouldn't close that door. Closing that door closes it permanently, so that it can't be opened again. Such a policy makes it impossible to use throwing violation handlers and programmatic recovery. That's much worse than a vast majority of WG21 not needing to do that, it means that the user communities who would like to do that just can't, the possibility is taken away.

This hurts people who know what they are doing, people who know what's right for their domain, and have a requirement that they should do whatever they can to avoid abrupt program terminations since they are ugly and user-unfriendly, and unnecessary. WG21 shouldn't be a bunch of people legislating against a use case they don't personally run into in their domains, or a use case they mostly philosophically oppose, mostly because it's not relevant in their domain, or is unacceptable in their domain. It's relevant in many others, especially ones that are underrepresented in WG21, programs that have non-expert end users. For such users, abruptly terminating a program is the worst thing you could ever do, and that has direct business impact, those users will vote with their feet and use applications that don't just vanish from their screens.